W. Shakespeare

Categories and functors: much ado about nothing?

Bachelor thesis

26 April 1564

Thesis supervisors: prof.dr. G.A. Bredero
dr. J. van den Vondel

Leiden University
Mathematical Institute

Contents

Chapter 1

Introduction

This study expands on an article written by A. Rumpf and H. Kaul, where they develop a model to optimize the social accessibility of a transit network, under the constraint that user costs do not increase too much and that there is no need for purchasing more vehicles or redrawing buslines. This results in a new schedule that can immediatly be implemented with minimal effort, thus increasing the likelihood it will be. Their approach consists of reallocating vehicles between lines, consequently altering line frequencies and thus the distance one can travel within a reasonable time.

This model uses a transit network, a set of community centers as origins, and a set of facility locations as destinations. They define an accessibility metric for a given origin region, putting a numerical value on how accessible the facilities are from that region, higher being better. The model then tries to maximize the worst regions, thus making the social access more equitable. They also develop an algorithm to solve it, and then use it on the real transit network of the Chicago area. They also run computational trials on an artificial network.

Their definition of social access stems from the fact that some people are too poor to have a car, thus limiting their mobility. Since wealth is generally not uniformly distributed over a city like Chicago, some communities being significantly wealthier than others, and the poorer regions generally having less healthcare facilities, not having a car significantly limits a persons access to healthcare. The group with limited mobility is assumed to be quite small, this is the main reason for the constraint where user costs can not icrease too much. We formulate this in the following assumption:

Assumption 1 *The group of people the model is made for is very small compared to daily travelers. Therfore, this small group will have no effect on overcrowdedness or other usage related metrics.*

In this study, we use a different cause of limited mobility for a person, not being able to walk or bike long distances. This applies mainly to (temporarily) disabled people and the elderly, which are groups that are more likely to need healthcare. We also note that the Chicago area used in [RK21] is very large, the total area can include both Rotterdam, The Hague and everything in between.

This study uses a much smaller study area, namely the municipality of Leiden and its bus system. This area will be divided into community regions in two ways. Firstly by the PC4 regions originating from the postcode system used in the Netherlands. It will also be divided by Voronoi regions to simulate everyone using their nearest bus stop, instead of one that is further away.

Both these methods result in smaller regions and thus in more accurate results at the cost of larger computing times. This study aims to find out how large this penalty in computing time is, relative to the increase in performance. Resulting schedules will also be compared to see if and how they differ.

Structure of this Thesis In Section 2 we introduce some mathematical background used, namely Voronoi regions and two neighborhood search algorithms: Tabu search and Simulated Annealing. In Section 3 we lay out the general model developed by Rumpf and Kaul, as well as their application on the transit system of Chicago. In Section 4 we explain our application of the model on the bus system of Leiden and the chosen parameters. Section 5 contains the setup of our experiment, with results in Section 6 and discussion in Section 7. Appendix A contains pseudocode for the algorithm explained in Section 3 and Appendix B contains data used in Section 4.

Chapter 2

Relevant mathematics

2.1 Voronoi

Following the definition in [Møl94], a voronoi diagram is given by a set $\Phi = \{x_i\}$ of weightpoints in a metric space X. Each weightpoint generates a region

$$C(x_i|\Phi) = \{y \in X : d(x_i, y) \leq d(x_j, y) \ \ \forall x_j \in \Phi\},$$

where d denotes some distance function. It follows that each voronoi region $C(x_i|\Phi)$ contains all points that have x_i as nearest weightpoint.

Applications of voronoi diagrams can be found in many fields. Consider for example the placement of automated external defibrillator (AED) boxes in a city. An even spread of these boxes would theoretically save the most people. The positions of these boxes could be used as weightpoints, with the euclidian distance in $\mathbb{R}^2$. This results in a voronoi diagram where smaller regions represent boxes that are relativily close to each other, while larger regions represent boxes that are far from others. This information could be used to move or add boxes, and thus getting a more even distribution.

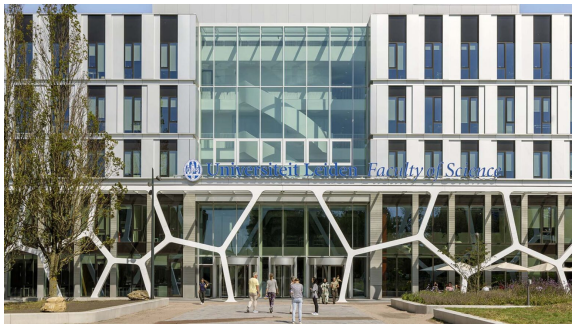

Figure 2.1: Entrance of Gorlaeus building of Leiden University, with voronoi diagram as decoration. Picture taken from [Swe25].

2.2 Neighborhood search algorithms

Given a solution space S and an objective function $f : S \to \mathbb{R}$, a general optimization problem can be $\max_{s \in S} f(s)$. Given large enough solution spaces, or computationally heavy objective functions, evaluating every $s \in S$ can become too cumbersome. To avoid calculating all solutions, a *neighborhood search* (NS) could be done.

A general NS algorithm needs a solution space S and objective f, as well as predefined *moves* N. Like [Glo89], we define a move m to consist of a mapping defined on a subset $S(m) \subseteq S$, $m : S(m) \to S$. These moves allow the transition between different solution, thus the algorithm can 'walk' from a starting solution s' to an optimal solution. For all solutions $s \in S$, a set $N(s) = \{m \in N : s \in S(m)\}$ can be defined, containing all possible moves for s. This set $N(s)$ is called the neighborhood of s.

A simple example of an NS algorithm is the so-called *Hill climb heuristic*, where each iteration looks at all nearby positions and moves to the highest one. When there is no higher position than the current one, the algorithm terminates. This algorithm has the following pseudocode

HILL CLIMB HEURISTIC

Select initial $s \in S$.
loop
    Compute $N(s)$ and $f(m(s))$ for all $m \in N(s)$.
    Find $s^* = m^*(s) \in N(s)$ such that $f(m^*(s)) \geq f(m(s))$ for all $m(s) \in N(s)$.
    if $f(s^*) > f(s)$ **then**
        Set $s = s^*$.
    else
        s is a local optima, **STOP**.

We will use this algorithm to find the highest value in the following grid, starting in the lower left corner. We define a move to be one step in any (orthogonal or diagonal) direction.

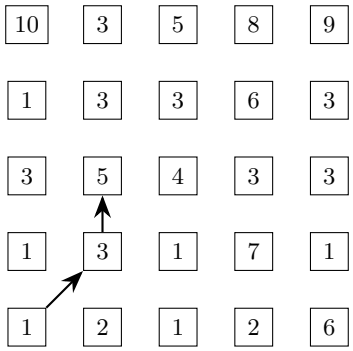

Figure 2.2: Hill climb heuristic in a grid. Optimal value (10) is not achieved.

Note that we get stuck in a cell with value 5, while there are multiple cells with higher values. The cells near our endpoint is higher than its neighbors, we call this a *local optimum*. To avoid getting stuck in local optima, many methods haven been developed. We dicuss two of those methods, both relying on allowing suboptimal moves. Just like NS, these are heuristic approaches to optimization problems.

2.2.1 Tabu search

Tabu Search (TS) is an NS algorithm that avoids getting stuck early by using a subset $M \subset N$ containing *tabu moves*, or forbidden moves. The process of adding or removing moves to or from M can be defined in many ways, we will go over the ones used in this study. An example of general TS pseudocode is the following.

TABU SEARCH

Select initial $s \in S$, let best known solution $s^* = s$.
Set $M = \emptyset$ and iteration counter $k = 0$.
loop
    Compute $N(s)$ and $f(m(s))$ for all $m \in N(s) \setminus M$.
    if $N(s) \setminus M = \emptyset$ **then STOP**.
    Set $k = k + 1$.
    Find $m_k(s) \in N(s) \setminus M$ such that $f(m_k(s)) \geq f(m(s))$ for all $m(s) \in N(s) \setminus M$.
    Set $s = m_k(s)$.
    if $f(s) > f(s^*)$ **then**
        Set $s^* = s$. ▷ *Update best known solution*
    if A chosen number of iterations is reached, in total or since last improvement **then**
        STOP
    Update M.

Although it resembles the previous hill climb heuristic, it differs in some subtle ways. The first thing we note is that line 7 does not compare neighboring solutions to the current solution s. This enables the algorithm to walk away from a local optimum, while still remembering the best known solution for later.

The algorithm also keeps track of how long it has been running, terminating in line 11 when improving takes too long. This is to ensure it does not walk in circles forever.

In line 13 we 'update M', the rules we use to do this are called *tabu tenures*. We will look at some tenures that can be used. One way is to remember some or all of the previous solutions that are visited. Moves that go to those solutions are tabu, others are not.

Another way is to remember what kind of moves have been done recently. In our example of Figure 2.2, the moves are (up-and-right) and (up). We could update M by making the reversal of these moves tabu for a number h of iterations. The set M would then be defined by $M = \{m^{-1} : m = m_l \text{ for } l > k - h\}$, where m^{-1} is the inverse of m, $m^{-1}(m(s)) = s$. The tabu moves in our example would then be (down-and-left) and (down). By not allowing the reversal of moves, the algorithm looks to diversify the evaluated solutions agressively. Because these moves become non-tabu after a relatively small number of iterations, we call this the *Short Term Memory* (STM).

We can also define *Long Term Memory* (LTM) in multiple ways. One of which is by remembering second-best solutions that were not chosen during an iteration. These could be revisited if the algorithm does not improve for a set amount of iterations (line 11).

If we say moves can not be inverted for $h = 2$ iterations, we take a maximum of $K = 12$ total and $I = 4$ non-improvent iterations and apply the TS to the grid in Figure 2.2, we get the path displayed in Figure 2.3.

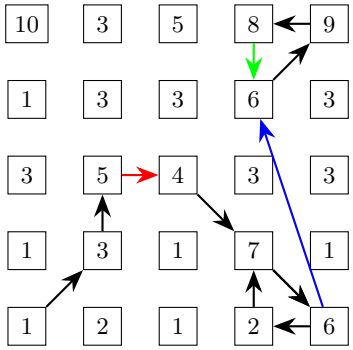

Figure 2.3: TS heuristic in a grid. The first 'downhill' move is marked red, this is not immediately reversed due to the STM tenure. The search then chooses the cell with value 7 and remembers the cell valued 6 in LTM. It loops in the lower left corner until 4 iterations without improvement force the revisiting of the cell valued 6, marked in blue. The algorithm terminates after the iteration marked in green, with the best known solution having value 9.

Note that this path still does not give the optimal value. In this case the optimal value would probably be achieved by increasing tenure h, forcing less loops to be walked. Exactly knowing which tenure rules and associated parameters (in our case h, K, I) is impossible. Through trial and error we can come close, but Tabu Search will always be a heuristic approach.

2.2.2 Simulated annealing

Where TS agressively searches for different options, *Simulated Annealing* (SA) gently moves around, believing the final local optimum it visits to be close in value to a global optimum. It is named after a technique in metallurgy where a material is heated and cooled in a slow, controlled fashion, resulting in a change in physical properties.

Just like the hill climb heuristic and TS, each iteration generates neighbors $N(s)$ of current solution s. Instead of choosing the neighbor with optimal value, it picks one solution s^* at random. It then compares the two solutions by defining the difference in *Energy* (SA's term for the objective value)

$$\delta E = f(s^*) - f(s).$$

If δE is positive, neighbor s^* is a better solution than s, so SA always makes the move to s^*. If δE is negative, SA makes the move with probability $\exp(\delta E/T)$, where T is the temparture of the model. A small decrease in energy or a large temperature make the probability of making sub-optimal moves higher. After each iteration, a cooling schedule is used to lower T for the next iteration. This stimulates that the algorithm can make lots of suboptimal moves at the start, and almost no suboptimal moves towards the end, with the hope that it is near a global maximum after some time. One example of pseudocode is the following

SIMULATED ANNEALING

```
1:  Select initial s ∈ S, set T = T_0.
2:  repeat
3:      Compute N(s).
4:      Pick s* = m*(s) ∈ N(s) at random.
5:      Set δE = f(s*) − f(s).
6:      if δE > 0 then
7:          Set s = s*.
8:      else
9:          Set r ∼ U[0,1].          ▷ random variable drawn uniformly form [0,1]
10:         if r < exp(δE/T) then
11:             Set s = s*.
12:     Cool T
13: until T < T'
```

There is also a choice to be made about the cooling procedure at line 12. Common choices can be to multiply T by some other parameter α, decreasing the temperature by say 5% per iteration, or letting T decrease linearly. Cooling can also change dinamically while running. Different choices in cooling procedure influence how erratic the alogrithm behaves during runtime.

The starting and ending temperatures T_0, T' can also be chosen, where total runtime, and energy of a given solution need to be considered.

As SA is random, exact paths for the grid in Figure 2.2 can not be made. To illustrate the random nature of SA, we apply it to the grid with the same starting position (bottom left) for ten runs. The following parameters are used: starting temperature $T_0 = 10$, ending temperature $T' = 0.5$ and we cool the system by multiplying with $\alpha = 0.95$ after eacht iteration. The results are shown in Figure 2.4. The algorithm terminates in the values 7 (bottom right), 8 and 9 (top right) each three times and the 10 in the top left a single time.

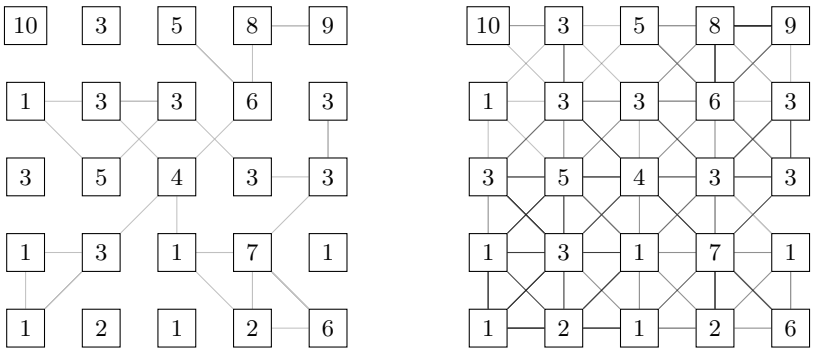

Figure 2.4: SA heuristic on a grid, chosen parameters are $T_0 = 10$, $\alpha = 0.95$, $T' = 0.5$, with $T := T \cdot \alpha$ as the cooling method . Left grid displays a single run of the SA algorithm, it walks seemingly at random until terminating at the cell valued 8 in the top right. The right grid displays ten different runs of the algorithm. Terminal values are: 7 in bottom right, 8 and 9 top right each three times and the 10 in the top left a single time.

Chapter 3

General model

This section summarizes the general model and algorithm developed by Rumpf and Kaul in [RK21]. As the model and algorithm is made by them, a more detailed explanation can be found in their paper. In this section we will give a summary so the later parts of this study can be understood.

3.1 SAMP

The algorithm made by Rumpf and Kaul in [RK21] was made to solve the *social access maximization problem* (SAMP). The formulation of the problem is very general as to be aplicable to many situations.

The network is made up of considered transit lines in set L. Here, each transit line $l \in L$ is a series of stops with a given travel time between each stop. Each line can have a type of vehicle assigned to it, so a network with different modes (like trains and busses) is possible.

Each line l has an amount of vehicles, called fleet size, assigned $y_l \in \mathbb{N}$ to it. These vehicles are allocated for the entire time horizon τ. Each line also has a circuit time τ_l, being the time it takes to travel a single loop of its schedule. After its circuit time is elapsed, the vehicle starts its next trip. Overall frequency of a line is thus dependent on amount of vehicles y_l and circuit time τ_l, it is given by

$$f_l = \frac{y_l \cdot \tau}{\tau_l}.$$

Frequency is derived from fleet size instead of the other way around, because operating costs are mostly dependent on fleet size. This makes schedule vector y, containing all fleet sizes y_l, the decision variable for the model.

The initial schedule is denoted as y^*. The model has upper and lower bounds $y_l^{\min} \leq y_l \leq y_l^{\max}$ for all $l \in L$. These can be set to 0 and infinity if no restriction is needed, or both to y_l^* if no change is wanted.

To simulate people traveling in the network, a user flow vector x is used. User flow represents how busy parts of the network are, which is influenced by the schedule y and found by the TransitAssignment(y) function, which will be explained further in Section 3.3. The initial user flow vector is denoted x^* and is found by TransitAssignment(y^*).

Other restraints of the model are given by operator and user costs with upper bounds $B_{operator}$, B_{user} respectively. Operating costs are mostly made up by monetary costs of running the transit network and monetary gain of users paying to use the service. User costs are made up of difficulty in traversing the network, be that in monetary cost or time spent in the system. More on these costs in Section 3.4.

Let Access(y) be a function describing the social access objective. This value only depends on the design of the network y and not on user flow x, because the model is designed for a small group of individuals following Assumption 1 Different choices for Access(y) are given in Section 3.5.

3.2 Network structure

The model uses a representation of the transit network to calculate travel time and route choices. For this, a directed graph $G = (V, A)$ is used. V is the set of all relevant nodes, be that origin,

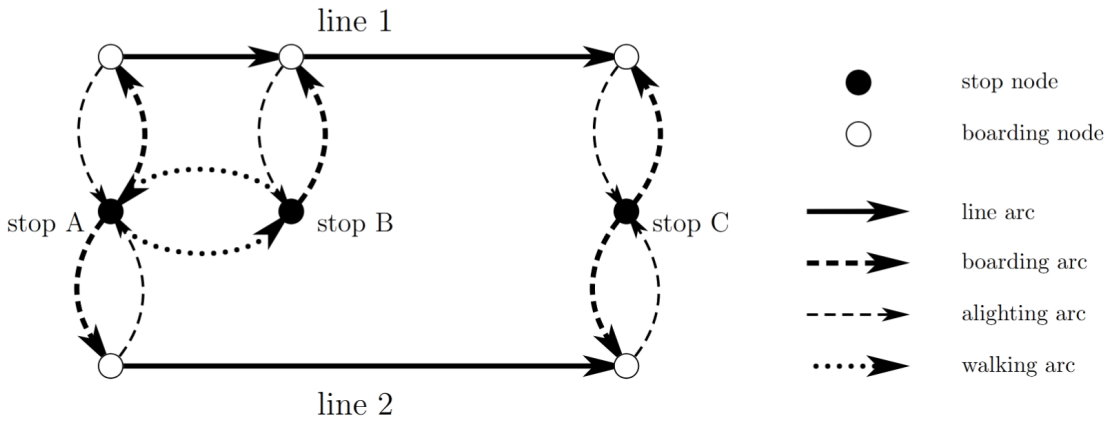

Figure 3.1: Example of network representation containing two lines and three stops. Each stop has nodes in V_{stop}. Line 1 stops at all stops and thus has three boarding nodes, line 2 only has boarding nodes at stop A and stop C. Boarding nodes are conected to their stops with boarding arcs and alighting arcs. Boarding nodes are connected by line arcs if a transit line connects their respective stops. Stop A and stop B are connected by walking arc, because they are close enough. Figure taken from [RK21].

destination, transit stop nodes. A is the set of arrows linking the nodes together, these represent different modes of travel like walking, bus or train riding.

The node set is partitioned in subsets, each containing specific nodes. Let $V_{stop} \subset V$ contian nodes representing the physical location of transit stops. Let $V_{board} \subset V$ be the set of boarding nodes. Each stop node $i \in V_{stop}$ has boarding nodes $j \in V_{board}$ for every line $l \in L$ that stops at that stop. Boarding nodes are used to distinguish between transit lines. Let $V^+, V^- \subset V$ be origin and destination nodes, respectively. These represent the physical places where trips begin and end. To calculate social access, subsets $\tilde{V}^+, \tilde{V}^- \subset V$ are added, representing community and facility nodes, respectively.

Just like the node set, the arc set A is partitioned in subsets. Let $A_{line} \subset A$ be the arcs that represent the journey made by transit lines and let $A_{board}, A_{alight} \subset A$ be the arcs representing entering or leaving a transit vehicle at a certain stop. Let $A_{walking} \subset A$ be walking arcs, connecting different nodes if their physical distance is walkable.

3.3 Transit Assignment

The transit assignment model, TransitAssignment(y), models the way daily users interact with a given schedule y. It assumes users try to take their optimal route, wich may not be the fastest due to overcrowdedness. The flow produced by the model is called *user-optimal flow* and is often less optimal then when all flow is controlled directly.

There are many such models, but Rumpf and Kaul choose the nonlinear model by Spiess and Florian in [SF89]. This model increases arc cost based on crowdedness, discouraging flows from exceeding line capacity. The user-optimal flow output can not be changed in a way that individual users benefit from changes in only their route. The model is relatively simple, a desired property, as the model is run many times during the algorithm.

3.4 Costs

Generally, operator costs are monetary costs relevant to running the transit system. Things like purchasing and maintaining vehicles, employing drivers and other staff and other costs. In this study, we restrict the total number of busses to not increase. This gives no reason to purchase new vehicles or employ more drivers, so we assume that operator costs will not increase with a new schedule. This means that we can omit the operator cost constraint from the model.

User costs are time costs made by using the system. Things like travel time, wait time, number of transfers and discomfort. All the user costs can be weighted based on transfer mode, 5 minutes of walking may weigh more than 5 minutes of bus riding for example. For each $(i, j) \in A$ a flow amount x_{ij} and cost c_{ij} can be implemented. Flow amount represents the amount of users taking that arc, while cost represents some penalty in using that arc, like time needed to traverse it. The user cost function takes the form

$$\text{UserCost}(x, \omega) = \theta_1 \sum_{ij \in A_{line}} c_{ij} x_{ij} + \theta_2 \sum_{ij \in A_{walk}} c_{ij} x_{ij} + \theta_3 \omega,$$

where $\omega \in \mathbb{R}$ is the total time spent waiting at busstops by all users combined.

Due to Assumption 1 we do not want the total user cost to increase too much, so we introduce a parameter ε to give a constant upper bound

$$\text{UserCost}(x, \omega) \leq (1 + \varepsilon)\text{UserCost}(x^*, \omega^*).$$

When choosing ε, a balance needs to be found between a large increase, giving a larger set of possible solutions and possibly a more optimal one at the cost of daily user experience.

3.5 Access

The objective function Access(y) should describe how easy it is to travel from a general community to a general facility, with a focus on equity of access. To do this, a metric is made to measure the accessibility for a specific community $i \in \tilde{V}^+$.

The chosen metric $A_i(y)$ is *gravity-based*, meaning that the attractiveness of facilities reduces gradually as distance increases. An alternative is a discrete distance based approach, where facilities $j \in \tilde{V}^-$ are considered only if they are within a certain distance from the source community i. This approach is not preferred, as facilities that lie just inside the considered radius are weighted equally to facilities that are nextdoor, and facilities that lie just a bit further are not considered at all. This is solved by including a distance function $d_{ij}(y)$, measuring traveltime within G from i to j, and a gravity decay parameter $\beta > 0$. A larger β will result in a higher penalty for further travel.

As the users can choose between facilities, a competition-related metric $F_j(y)$ is used to find the crowdedness of facilities. It is defined as

$$F_j(y) = \sum_{k \in \tilde{V}^+} P_k d_{kj}^{-\beta}(y) \quad \forall j \in \tilde{V}^-$$

where P_k is the population of community k and $d_{kj}^{-\beta}(y)$ is the previously mentioned gravity decay. This crowdedness can be compared to the quality S_j of a facility, usually taken as some metric of patient capacity.

The accessibility metric for a given region is then defined as

$$A_i(y) = \sum_{j \in \tilde{V}^-} \frac{S_j d_{ij}^{-\beta}(y)}{F_j(y)} \quad \forall i \in \tilde{V}^+.$$

This metric was first developed in [Wei76] to decribe differential access to employement opportunities, but can also be used for other competition based access cases.

Since the goal is to create more equity in accisibility, not all communities should be included in the overall objective function. Instead, the communities with the worst accessibility should be improved. By maximizing the minimum communities, a more equitable scenario is achieved. The model parameter $\mathcal{K}$ is introduced to achieve this. The objective function is then defined as

$$\text{Access}(y) = \sum_{\substack{\mathcal{K} \text{ minimum elements of} \\ \{A_i(y) | i \in \tilde{V}^+\}}} A_i(y).$$

Note that the objective function will be evaluated many times, and that the $\mathcal{K}$ worst communities are dependent on the y of the current iteration, so the evaluated and improved communities changes while the algorithm is running.

The parameter $\mathcal{K}$ needs to be chosen between 1 and $|\tilde{V}^+|$, where extreme values are not benificial to the outcome. Low values for $\mathcal{K}$, like 1, may result in minor improvements to a single community with no regard for the relatively large effect on other communities. High values, like $|\tilde{V}^+|$, may result in making large improvements to the highly accessible communities at a cost of making the less eccessible communities even worse, as long as the total accessibility increases.

3.6 The model

The SAMP model takes the shape of the following nonlinear program

$$\max_y \ \text{Access}(y) \tag{3.1}$$

$$\text{s.t.} \ \sum_{l \in L} y_l \leq \sum_{l \in L} y_l^* \tag{3.2}$$

$$y_l^{\min} \leq y_l \leq y_l^{\max} \quad \forall l \in L \tag{3.3}$$

$$y_l \in \mathbb{N} \qquad \forall l \in L \tag{3.4}$$

$$x = \text{TransitAssignment}(y) \tag{3.5}$$

$$\text{UserCost}(x, \omega) \leq (1 + \varepsilon)\text{UserCost}(x^*, \omega^*) \tag{3.6}$$

Where Eq. (3.1) is the objective function described in section 3.5, Eq. (3.2) constrains the total number of vehicles to not increase from the initial amount, Eq. (3.3) ensures upper and lower bounds for vehicle allocation, Eq. (3.4) ensures a whole number of vehicles is allocated to each line, Eq. (3.5) defines user flow as discussed in section 3.3 and Eq. (3.6) gives an upper bound to the increase in user cost.

3.7 General algorithm

The SAMP model can be solved heuristically with the following algorithm. It is a combination of *Tabu search* (TS) and *Simmulated annealing* (SA), two versions of neigborhood search mentioned in Sections 2.2.1 and 2.2.2.

Pure neighborhood search runs the risk of getting stuck in local optima, so both SA and TS try to avoid this by sometimes making suboptimal moves. For current solution s we have a neighborhood $N(s)$ of solutions reachable within one move m. SA picks a random neighbor $s' \in N(s)$ and goes there with probability

$$\mathbb{P}(\text{accept} \ \ s') = \begin{cases} 1 & \text{if } f(s') > f(s) \\ \exp\left(\frac{f(s') - f(s)}{T}\right) & \text{else.} \end{cases} \tag{3.7}$$

Where f is the objective function that we want to maximize and T is a model parameter called temperature that decreases after each iteration. TS always picks the best neighbor, even if that neighbor has a lower objective value. It then stores the last neighbor in *short term memory* (STM) to make that neighbor *tabu* or forbidden. STM can also be used to store the reversal of recent moves, forcing the algorithm to 'walk away' from recent solutions. It also stores notable good solutions that were not visited or local maxima in *long term memory* (LTM) to be revisited later. How long moves and solutions are stored in memory depends on *tabu tenures*. The algorithm used for this study uses a hybrid between the two. Full pseudocode can be found in Appendix A.

Using the current transit schedule as a starting point, a neighborhood search is conducted. The neighborhood $N(s)$ of a solution s is defined as all other solutions that are reachable within one move m. A move, in this instance, is defined as swapping one allocated vehicle from one line to another for the entire time horizon. Each SWAP consist of an ADD move and a DROP move, where a vehicle is added or dropped from a line, respectively.

During each iteration, the two best neighbors s_1', s_2' are determined. If the best neighbor is an improvement over the current solution, that move is made and the next iteration starts. Reset the inner nonimprovement counter, save the chosen ADD and DROP move in STM.

If there is no improvement, the SA criterion (3.7) is used and inner nonimprovement counter is increased. The best neighbor is chosen with probability $\exp(f(s_1') - f(s)/T)$, and s_2' is saved in LTM. If s_1' is not chosen, solution s is kept as current solution and s_1' is saved in LTM. If a move is made, save the ADD and DROP moves in STM.

If the inner nonimprovement counter has met its maximum, reset it, increase the outer nonimprovemnt counter and move to a random solution in LTM. If the outer nonimprovement counter has met its maximum, reset tabu tenures, thereby dropping moves from STM. After each iteration, apply a cooling schedule to T.

After a preset amount of iterations, an exhaustive local search is done for the best known solution. This ensures that the final solution is locally optimal.

3.8 Chicago application

In their paper, Rumpf and Kaul applied their model to the transitsystem of Chicago [RK21]. In their application, a transit stop set V_{stop} with 997 different stops was used. These stops were also used as origin-destination (OD) pairs for day-to-day travel volume. Community nodes $\tilde{V}^+$ were defined using the $|\tilde{V}^+| = 77$ Chicago community areas, where node position was taken as population-weighted centroids, based on 2010 cencus data. Facility nodes $\tilde{V}^-$ were the primary care facilities, of which $|\tilde{V}^-| = 119$ are within the Chicago area. Each facility was given a quality of $S_j = 1, \ \forall j \in \tilde{V}^-$. Walking arcs were created when different locations were 0.75 miles (approximately 1.2 km) within eachother, walking speed was set at 4 ft/sec (approximately 4.4 km/h).

Transit lines were aquired from General Transit Feed Specification (GTFS) data. It includes 126 bus lines and 8 train lines. It was also used to estimate the current number of vehicles y_l^* for each line $l \in L$. The time horizon τ was taken as 24-hour period. It was assumed a single type of bus was used, with capacity $b_l = 39$ for all buslines, and a single type of train with capacity $b_l = 228$ for all trainlines. Fleet bounds $y_l^{\min}, y_l^{\max}$ were set to equal y_l^* for all train lines to force these to remain the same. For bus lines $y_l^{\min}$ was set to an amount to achieve at least one arrival per 30 minutes, no maximum $y_l^{\max}$ was set.

Their computational trials also used express lines, existing bus lines only servicing approximately 20% of their initial stops. With less stops, a lower circuit completion time τ^l could be achieved, thus increasing their effective frequency f_l. The stops that remained were in highly accessible communities or in badly accessible communities, so the lines could connect badly accesibble communities to facilities. These express lines had initial fleet sizes of $y_l^* = 0$.

The algorithm was run for 500 iterations, followed by a neighborhood search of the best found solution to ensure local optimality. The access function in Section 3.5 was evaluated with gravitational decay parameter $\beta = 1.0$ and community count of $\mathcal{K} = 8$ (approximately 10.39% of 77 communities). The user cost function in Section 3.4 used equal mode weights $\theta_1 = \theta_2 = \theta_3 = 1$ and maximum relative increase $\varepsilon = 0.01$.

They also ran trials where either user cost constraints were ignored, only allowing vehicle swaps between lines and their express lines, and excluding express lines.

Chapter 4

Case Study: Leiden

This section will discuss our implementation of the same model on Leiden. It will also go over the different choices of parameters and motivate these choices.

The municipality of Leiden is much smaller than the Chicago area. This naturally gives a lower amount of busstops, buslines, health facilities and so forth. Although there are multiple train stations in Leiden, the trainsystem is not taken into account for this study.

4.1 Transit system

As Qbuzz, the company running the bussytem in Leiden, operates in a greater region, most of their buslines lie partially or completely outside of the study area. For this study, only buslines that travel at least partially through the study area are considered. A busline usually drives a lineair route between two points, stopping at bus stops in between. For the purposes of this study, the back and forth trip is considered a complete circuit.

Buslines that service busstops outside the study area are modeled to have all stops within the area and a single stop at their turnaround point. For example, line 20 between Leiden Centraal and Noordwijk Vuurtorenplein services 26 stops in total, 4 of which are within the study area. It is modeled to have stop nine times per circuit, twice per stop within the municipality and once at Vuurtorenplein. This way of modeling ensures the circuit time τ_l works correctly for each line l.

One notable exception to this method are the linking of lines 1, 3 and 4. Their scheduled routes end where the next route begins. Line 1 travels between De Vink and Leyhof, line 3 between Leyhof and Merenwijk and line 4 between Merenwijk and De Vink. The vehicles allocated to these lines currently continue the next line after completing their given trip. We will keep this planning choice and thus model these lines as two lines, one for each direction.

Since Leiden is much smaller than the Chicago area, the implementation of express lines was excluded.

After set of buslines L was selected, a schedule was retreived from [Qbu25]. The chosen schedule was for monday the 10th of february of 2025, between 14:00 and 15:00 in the afternoon. This weekday and timeframe was chosen as a ordinary, non-rush hour time, that represents ordinary bus travel.

The infromation given by a schedule is average traveltime between stops, wait time at a stop and amount of busses per line per hour. The used algorithm takes busses per line per time-horizon τ and averages this out to arrive at the frequency of a line, where more busses allocated over a time-horizon would mean a higher frequency of service. The circuit time τ_l of a line l is the time needed to complete a round trip, after which it can be used to complete a new trip. Buslines with a lower circuit time can thus get a appropriate frequency while having a lower amount of busses allocated to them, relative to lines that have longer circuit times.

Names of- and traveltime between busstops can be retrieved from the schedule, giving a transit-stop set V_{stop} of 121 different busstops, of which 100 are within the study area. As most bus stops consist of a pair of physical busstops, one for each direction, the midpoint was taken to represent the physical pair in the graph. Distances to other points, like other busstops or community centers, can be determined and walking arcs can be connected accordingly.

For simplicity it is assumed that the operator uses a single type of bus, the Iveco Crossway EV 13m, with a capacity of $b_l = 50$ for all lines $l \in L$.

4.2 Facility and Community nodes

Facility nodes $\tilde{V}^-$ are defined by the general practitioners (GP) offices of Leiden. Taken from [Ned25] a total amount of $|\tilde{V}^-| = 37$ are used. The amount of full-time employed doctors that work at a facility j is taken as the quality S_j of that facility.

For this study, we compare two different methods of defining community nodes $\tilde{V}^+$. The first method concerns itself with postcoderegions in the Netherlands. Each adress in the Netherlands has a postcode, consisting of four numbers and two letters. If one only uses the four numbers, the Netherlands can be divided into pc4 regions. There are $|\tilde{V}^+_{\text{pc4}}| = 16$ different pc4 regions within the municipality. Center node location was taken as their middlepoint and each node was assigned population data taken from [All25].

These pc4 regions are still quite large and all, non-day to day trips are simulated to start at the center node to then walk to a bus stop. This may imply that busstops far from the middlepoint of a region are used less then in reality. To remedy this, another way of defining community nodes is found.

According to [Wan22], travelers prefer to take a bus at the nearest bus stop, even when that involves a transfer instead of a direct route. To apply this effect on our study, the regions we choose should reflect trips that start at the closest bus stop. Voronoi diagrams, as defined in Section 2.1, are the ideal method of dividing a plane in sections where each section contains one weightpoint and all points that have that weighpoint as the nearest weightpoint.

Using this method, with busstops as weightpoints, we can divide the municipality in $|\tilde{V}^-_{\text{vor}}| = 100$ regions. We have one region less than the number of busstops, because one busstop lies outside the municipality. We estimate the population data for the voronoi regions by looking at the overlap between each voronoi region and the pc4 regions, and assuming the known population of a pc4 region is distributed evenly over its surface. Let

$$\begin{aligned}
\tilde{V}^-_{\text{vor}} &\text{ set of voronoi regions,}\\
\tilde{V}^+_{\text{pc4}} &\text{ set of postcode regions,}\\
s_v &\text{ surface area for } v \in V^-_{\text{vor}},\\
d_p &\text{ population density for } p \in \tilde{V}^+_{\text{pc4}},\\
f(v,p) &\text{ fraction of } v \text{ that lies in } p.
\end{aligned}$$

Then, the population of a voronoi region v is given by

$$s_v \cdot \sum_{p \in \tilde{V}^+_{\text{pc4}}} d_p f(v,p).$$

4.3 Travel demand

No actual travel data could be aquired, so this had to be estimated. As the schedule gives the frequencies of service, it also gives the total supply of bus transit. Assuming the supply is an accurate reflection of the demand of bus transit, all busses are roughly equally filled.

We therefore assume that each bus is halffull. The algorithm wants traveldemand between busstops, so this data was made by counting the amount of lines servicing a pair of bus-stops directly and multiplying this by the lines' frequency and the capacity of a bus. The resulting table can be found in Appendix B.2.

4.4 Chosen parameters

Since our study tries to improve accessibility for less mobile people, we want to limit the distances walked. Therefore the choice was made to connect nodes via walking arcs if they are within 250 meters of eachother, and walking speed was set at 4km/h. Since all day-to-day travel is simulated to begin and end at a busstop, this change only affects those travelers when walking between busstops for a transfer. It will have a much larger effect the group of less mobile people that the algorithm is made to optimize for.

The time horizon τ was set at 540 minutes, the tipical time for a GP to be open during a weekday. The schedule taken is one based on 60 minutes outside of rush-hour, and is thus

extrapolated to count for the whole day. This is slightly inaccurate, as more busses are used during rush-hour to account for the higher demand. Since the algorithm averages out both the travel-demand and the busses used, accounting for this inaccuracy would be undone by the algorithm.

Fleet bounds are set almost identically as in [RK21]. For all lines $l \in L$, no upper bound $y_l^{\max}$ is used. The lower bound $y_l^{\min}$ is set such that at least two arrivals are made per hour, unless the starting schedule has a lower frequency, in which case we use the fleet size of the starting schedule as a lower bound. Gravitational decay parameter $\beta = 1.0$, maximum relative user cost increase $\varepsilon = 0.01$ and mode weights $\theta_1 = \theta_2 = \theta_3 = 1$ are also kept the same as in the Chicago case.

Seeing as the amount of pc4 communities is quite small ($|\tilde{V}^+_{\text{pc4}}| = 16$), and two of these regions are very sparsely populated with no bus service, taking roughly 10% of this would not suffice. Thus the community count is set higher at $\mathcal{K}_{\text{pc4}} = 4$, or 25%. The voronoi regions do not have these issues, so their community count can be set at $\mathcal{K}_{\text{vor}} = 10$ to be in line with the Chicago case. **These parameters may be finetuned after running, just like in the paper by Rumpf**

Chapter 5

Explanation of experiment

Chapter 6

Results

Chapter 7

Discussion

Appendix A

Pseudocode

The pseudocode for the soluion algorithm discussed in section 3.7, directly taken from [RK21].

x represents the entirety of the flow vector. The vector $e_l \in \{0,1\}^{|L|}$ represents a vector with a 1 in coordinate l and 0 elsewhere. Sets $\tilde{N}^+, \tilde{N}^-, \tilde{N}^\times$ represent the ADD, DROP and SWAP neighbors kept as candidates after the first pass of the neighborhood search, respectively, while $N^+, N^-, N^\times$ are the second-pass candidates. Bounds $\tilde{N}_{\max}, N_{\max}$ are the required numbers of candidates to be gathered during each pass. $Q_{\text{in}}^{\max}, Q_{\text{out}}^{\max}$ are the inner and outer nonimprovement counter bounds, respectively.

// INITIALIZATION
  $y^{(0)} \leftarrow y^*$ ▷ *current real-world fleete sizes*
  $y_{\text{best}} \leftarrow y^{(0)}$ ▷ *best known solution and objective*
  BestAccess $\leftarrow$ Access$(y^{(0)})$ ▷ *best known objective*
  STM $\leftarrow \emptyset$ ▷ *tabu move list, equipped with tabu tenures*
  LTM $\leftarrow \emptyset$ ▷ *attractive solution set*
  $T \leftarrow T_0$ ▷ *simulated annealing temperature*
  $t \leftarrow t_0$ ▷ *tabu tenure*
  $Q_{\text{in}} \leftarrow 0$ ▷ *inner noninmrovement counter*
  $Q_{\text{out}} \leftarrow 0$ ▷ *outer noninmrovement counter*
for $k = 1, \ldots, K$ **do** ▷ *search for a predetermined number K of local moves*
  // NEIGHBORHOOD SEARCH
  $\tilde{N}^+ \leftarrow \emptyset$ ▷ *tentative ADD neighborhood*
  repeat ▷ *first pass (ADD)*
    $l \leftarrow$ random element of L not yet considered during this loop
    if $y^{(k)} + e_l$ satisfies vehicle limit bounds (3.1 - 3.3) **then** ▷ *consider only design-fesible moves*
      if $e_l \notin$ STM or Access$(y^{(k)} + e_l) >$ BestAccess **then**
        $\tilde{N}^+ \leftarrow \tilde{N}^+ \cup \{l\}$ ▷ *keep non-tabu candidates (unless improved best)*
  until $|\tilde{N}^+| \geq \tilde{N}_{\max}$ or all lines in L have been considered
  $N^+ \leftarrow \emptyset$ ▷ *final ADD neighborhood*
  repeat
    $l \leftarrow$ unprocessed element of $\tilde{N}^+$ with maximum Access$(y^{(k)} + e_l)$
    $x^{(k)} \leftarrow$ TransitAssignment$(y^{(k)} + e_l)$ ▷ *calculate user flows for given move*
    if $(y^{(k)} + e_l, x^{(k)})$ satisfy operator and user cost constraints (3.5 - 3.6) **then**
      $N^+ \leftarrow N^+ \cup \{l\}$ ▷ *keep only feasible candidates*
  until $|N^+| \geq N_{\max}$ or all lines in $\tilde{N}^+$ have been processed
  $\tilde{N}^- \leftarrow \emptyset$ ▷ *tentative DROP neighborhood*
  repeat ▷ *first pass (DROP)*
    $l \leftarrow$random element of L not yet considered during this loop
    if $y^{(k)} - e_l$ satisfies vehicle limit bounds (3.1 - 3.3) **then** ▷ *consider only design-fesible moves*
      if $-e_l \notin$ STM or Access$(y^{(k)} - e_l) >$ BestAccess **then**
        $\tilde{N}^- \leftarrow \tilde{N}^- \cup \{l\}$ ▷ *keep non-tabu candidates (unless improved best)*
  until $|\tilde{N}^-| \geq \tilde{N}_{\max}$ or all lines in L have been considered
  $N^- \leftarrow \emptyset$ ▷ *final DROP neighborhood*

repeat
  $l \leftarrow$ unprocessed element of $\tilde{N}^-$ with maximum Access$(y^{(k)} - e_l)$
  $x^{(k)} \leftarrow$ TransitAssignment$(y^{(k)} - e_l)$ ▷ *calculate user flows for given move*
  if $(y^{(k)} - e_l, x^{(k)})$ satisfy operator and user cost constraints (3.5 - 3.6) **then**
    $N^- \leftarrow N^- \cup \{l\}$ ▷ *keep only feasible candidates*
until $|N^-| \geq N_{\max}$ or all lines in $\tilde{N}^-$ have been processed
if $|N^+| = |N^-| = 0$ **then** ▷ *handle the event of a failed search*
  if bounds $\tilde{N}_{\max}$ were used in the previous neighborhood search **then**
    restart neighborhood search while ignoring the bounds $\tilde{N}_{\max}$
  else
    $Q_{\text{out}} \leftarrow Q_{\text{out}} + 1$ ▷ *work towards diversification*
    delete from STM the tabu rule with the shortest remaining tenure
    restart neighborhood search
$\tilde{N}^\times \leftarrow \{(m, l) \in N^- \times N^+ : z_m = z_l\}$ ▷ *set of all compatible DROP/ADD pairs*
$N^\times \leftarrow \emptyset$ ▷ *final SWAP neighborhood*
repeat
  $(m, l) \leftarrow$unprocessed element of $N^\times$ with maximum Access$(y^{(k)} - e_m)$+ Access $(y^{(k)} + e_l)$
  $x^{(k)} \leftarrow$ TransitAssignment$(y^{(k)} - e_m + e_l)$
  if $y^{(k)} - e_m + e_l, x^{(k)}$ satisfy operator and user cost constraints (3.5 - 3.6) **then**
    $N^\times \leftarrow N^\times \cup \{(m, l)\}$ ▷ *keep only feasible candidates*
until $|N^\times| \geq N_{\max}$ or all moves in $\tilde{N}^\times$ have been processed
$\tilde{y}^{(k,1)} \leftarrow$ element of $N^+ \cup N^- \cup N^t imes$ with greatest access ▷ *returning two best neighbors*
$\tilde{y}^{(k,2)} \leftarrow$ element of $N^+ \cup N^- \cup N^t imes$ with second-greatest access
// NEIGHBOR PROCESSING
if Access$(\tilde{y}^{(k,1)})$ - Access$(\tilde{y}^{(k)}) >$ **then**
  $Q_{\text{out}} \leftarrow 0$ ▷ *improvement iteration*
  $t \leftarrow t_0$
  $y^{(k,1)} \leftarrow \tilde{y}^{(k,1)}$ ▷ *move to improving neighbor*
  add rules with tenure t to STM to prevent undoing the DROP/ADD moves of $y^{(k,1)}$
  if Access$(\tilde{y}^{(k,1)})$¿ BestAccess **then**
    $y_{\text{best}} \leftarrow \tilde{y}^{(k,1)}$ ▷ *update best known solution*
    BestAccess $\leftarrow$ Access$(\tilde{y}^{(k,1)})$
else
  $Q_{\text{out}} \leftarrow Q_{\text{out}} + 1$ ▷ *nonimprovement iteration*
  $Q_{\text{in}} \leftarrow Q_{\text{in}} + 1$
  $r \sim U[0, 1]$ ▷ *random variable drawn uniformly from* $[0, 1]$
  if $r <$ exp$\{-[$Access$(y^{(k)}) -$ Access$(\tilde{y}^{(k,1)})]/T\}$ **then**
    increment t ▷ *SA criterion passed*
    $y^{(k+1)} \leftarrow \tilde{y}^{(k,1)}$ ▷ *move to non-improving neighbor*
    make undoing the DROP/ADD moves of $y^{(k+1)}$ tabu with tenure t
    $Q_{\text{in}} \leftarrow 0$
    LTM $\leftarrow$ LTM $\cup\{\tilde{y}^{(k,2)}\}$ ▷ *keep second-best neighbor as an attractive solution*
  else
    LTM $\leftarrow$ LTM $\cup\{\tilde{y}^{(k,1)}\}$ ▷ *keep best neighbor as an attractive solution*
// UPKEEP
if $Q_{\text{in}} \geq Q_{\text{in}}^{\max}$ **then**
  $Q_{\text{in}} \leftarrow 0$ ▷ *attempt to diversify*
  $Q_{\text{out}} \leftarrow Q_{\text{out}} + 1$
  $y \leftarrow$ randomly chosen element of LTM
  LTM $\leftarrow$ LTM $\setminus\{y\}$
  $y^{(k+1)} \leftarrow y$ ▷ *move to a random attractive solution*
  increment t
if $Q_{\text{out}} \geq Q_{\text{out}}^{\max}$ **then**
  $t \leftarrow t_0$ ▷ *attempt to intensify*
remove moves from STM whose tenures have elapsed
apply cooling schedule to T
return $(y_{\text{best}},$ BestAcess$)$

After the algortihm terminates and we obtain the best known solution, we conduct an exhaustive local search starting at y_{best}. In each iteration of this local search we consider every possible ADD, DROP, and SWAP move without considering tabu rules, always taking the best neighbor and terminating when no neighbors offer any improvement. The final result is guaranteed to be locally optimal.

Appendix B

Data used

All data used for this study. Community and travel data was created following methodology in Section 4. Other data, like bus-schedule, bus-stops and GP-office data, was only formatted after being aquired. All used `.csv`-files can be found on [Ock]. Note that data related to the bus-schedule and general practitioners may be outdated as these are prone to modification.

B.1 Communities

B.1.1 PC4

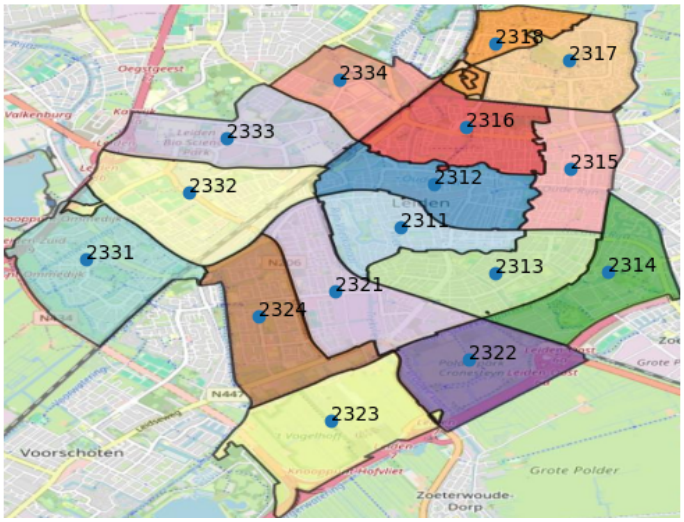

Figure B.1: Used PC4 regions. Community center points are based on rough centerpoint of each region.

Population and location data for PC4 regions. Location is based on the rough centerpoint of each region, coordinates can be found on [Ock]. Population is taken from [All25].

ID	Population	ID	Population
2311	11.355	2321	12.795
2312	15.260	2322	225
2313	12.375	2323	170
2314	6.200	2324	9.965
2315	8.715	2331	10.705
2316	10.725	2332	10.545
2317	10.720	2333	4.300
2318	3.245	2334	2.805

B.1.2 Voronoi

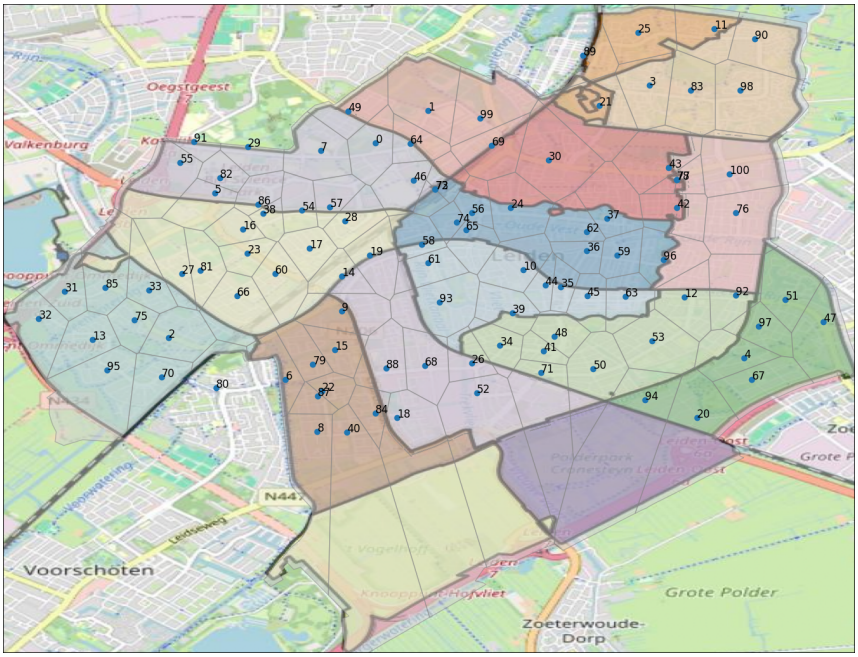

Figure B.2: Used voronoi regions. Center points are based on bus stops defining each region. PC4 border data is taken from [Sta25].

Population data for generated Voronoi regions, location data can be found on [Ock].

ID	Population	ID	Population	ID	Population	ID	Population
0	518	26	1125	52	3035	78	225
1	996	27	1160	53	2317	79	821
2	1145	28	696	54	496	80	220
3	1804	29	291	55	1096	81	795
4	1206	30	2955	56	1113	82	257
5	782	31	846	57	383	83	1998
6	863	32	1246	58	1531	84	726
7	672	33	944	59	1463	85	1542
8	1901	34	1146	60	1041	86	345
9	1654	35	870	61	1470	87	413
10	2281	36	1167	62	1670	88	1583
11	1418	37	2277	63	1765	89	578
12	1816	38	340	64	376	90	2240
13	1025	39	1681	65	1424	91	129
14	862	40	2039	66	1510	92	1969
15	895	41	586	67	1100	93	2971
16	537	42	1633	68	1633	94	1338
17	720	43	2201	69	1356	95	2625
18	2716	44	846	70	994	96	2073
19	1285	45	1407	71	2052	97	836
20	1108	46	520	72	658	98	3257
21	2521	47	1484	73	514	99	884
22	495	48	1037	74	732	100	2637
23	546	49	536	75	683		
24	2526	50	2220	76	2546		
25	1501	51	1160	77	483		

B.2 Travel data

Origin-Destination data estimated with the methodology described in Section 4. Columns O and D contain bus-stop ID's, column V contains travel volume from corresponding origin to destination over the entire time horizon of 540 minutes.

O	D	V	O	D	V	O	D	V	O	D	V	O	D	V
0	7	450	26	34	450	46	64	2250	68	52	450	93	61	1125
1	0	450	26	68	450	46	69	450	68	88	450	94	20	450
1	64	900	27	33	900	46	72	6300	69	30	450	94	112	450
1	99	450	27	81	450	47	51	2025	69	46	450	95	13	450
2	70	450	28	17	450	47	111	1125	70	80	900	95	70	450
3	21	450	28	54	450	47	120	450	70	95	450	96	42	450
4	67	450	28	72	450	48	34	450	71	35	1800	97	4	450
4	97	450	29	7	450	48	44	1800	71	48	1350	97	92	450
5	86	3600	29	91	450	48	71	450	71	50	450	98	83	450
5	102	450	30	21	450	49	64	1350	71	117	1800	99	1	450
5	103	450	30	43	1800	49	101	900	72	73	8550	99	108	450
5	113	900	30	46	1800	49	106	450	73	28	900	100	76	450
5	116	900	30	69	450	50	53	450	73	46	6300	101	49	900
5	118	900	31	32	450	50	71	900	73	57	900	102	5	450
6	79	450	31	85	450	51	47	1575	73	74	3825	103	5	450
6	80	900	32	13	450	51	92	2025	73	86	3600	104	55	450
6	87	450	32	31	450	52	71	450	74	10	2250	105	55	225
6	88	450	33	27	450	53	12	450	74	58	1575	106	49	450
7	1	450	33	75	450	53	50	900	75	2	450	107	80	450
7	29	450	33	85	450	54	28	450	76	47	450	108	99	450
8	40	450	34	26	450	54	38	450	77	78	450	109	91	450
9	14	450	34	41	450	55	82	900	77	100	450	110	77	450
9	15	450	35	36	450	55	91	225	77	110	450	111	47	1125
10	44	2250	35	44	450	55	104	450	78	42	450	112	94	450
10	65	2700	35	62	2250	55	105	225	78	77	900	113	5	900
11	90	450	35	71	1800	56	72	4275	79	6	450	114	43	450
12	51	1575	36	59	450	57	72	900	79	15	450	115	43	450
12	53	900	37	42	450	57	82	900	80	6	1350	116	5	900
12	63	1575	37	62	450	58	19	450	80	70	450	117	71	1800
12	97	450	38	16	450	58	61	1125	80	107	450	118	5	900
13	32	450	38	54	450	58	65	1575	81	27	900	119	43	900
13	95	450	39	45	1125	59	96	450	81	66	450	120	47	450
14	9	450	39	93	1125	60	23	450	82	55	900			
14	19	450	40	84	450	60	66	900	82	57	900			
15	9	450	41	48	450	61	58	1125	83	3	450			
15	79	450	42	37	450	61	93	1125	84	88	450			
16	23	450	42	78	900	62	24	1800	85	31	450			
16	38	450	43	30	1800	62	35	2250	85	33	450			
17	60	450	43	114	450	62	37	450	86	5	3600			
19	14	450	43	115	450	63	12	1575	86	72	3600			
19	58	450	43	119	900	63	45	1575	87	8	450			
20	67	450	44	10	2700	64	1	900	88	6	450			
20	94	450	44	35	900	64	46	2250	88	68	900			
21	30	450	44	45	450	64	49	1350	89	25	450			
21	89	450	44	48	900	65	56	4275	90	98	450			
23	16	450	45	39	1125	66	60	450	91	29	450			
23	60	450	45	44	450	66	81	900	91	55	225			
24	46	1800	45	63	1575	67	4	450	91	109	450			
24	62	1800	46	24	1800	67	20	450	92	12	2475			
25	11	450	46	30	1800	68	26	450	93	39	1125			

Bibliography

[All25] Allecijfers. May 1, 2025. URL: https://allecijfers.nl/postcode/.

[Glo89] Fred Glover. "Tabu Search - Part 1". In: *ORSA Journal on Computing* 1.3 (1989).

[Møl94] Jesper Møller. *Lectures on Random Voronoi Tessellations*. Springer New York, NY, 1994. DOI: https://doi.org/10.1007/978-1-4612-2652-9.

[Ned25] Patientenfederatie Nederland. May 2, 2025. URL: https://www.zorgkaartnederland.nl/huisartsenpraktijk/leiden.

[Ock] Jeroen Ockers. URL: https://github.com/Jerrie10/Bachelor-Thesis/tree/main/Data%20Used.

[Qbu25] Qbuzz. Feb. 3, 2025. URL: https://zhn.qbuzz.nl/?direction=1&date=2025-02-10.

[RK21] Adam Rumpf and Hemanshu Kaul. "A public transit network optimization model for equitable access to social services". eng. In: (2021).

[SF89] Heinz Spiess and Michael Florian. "Optimal strategies: A new assignment model for transit networks". In: *Transportation Research Part B: Methodological* 23.2 (1989), pp. 83–102. ISSN: 0191-2615. DOI: https://doi.org/10.1016/0191-2615(89)90034-9. URL: https://www.sciencedirect.com/science/article/pii/0191261589900349.

[Sta25] Central Bureau voor de Statistiek. Oct. 20, 2025. URL: https://public.opendatasoft.com/explore/assets/georef-netherlands-postcode-pc4/.

[Swe25] Sweco. Sept. 18, 2025. URL: https://www.sweco.nl/portfolio/het-gorlaeus-gebouw/.

[Wan22] M. Wang F.; Ye. "Optimization Method for Conventional Bus Stop Placementand the Bus Line Network Based on the Voronoi Diagram." In: *Sustainability* 14.7918 (2022). DOI: https://doi.org/10.3390/su14137918.

[Wei76] Jörgen W. Weibull. "An axiomatic approach to the measurement of accessibility". In: *Regional Science and Urban Economics* 6.4 (1976), pp. 357–379. ISSN: 0166-0462. DOI: https://doi.org/10.1016/0166-0462(76)90031-4. URL: https://www.sciencedirect.com/science/article/pii/0166046276900314.